

```
"""Voxface : squelette de backend pour avatars parlants.
```

```
Les étapes IA (voix, animation, rendu) sont simulées. Pour brancher un vrai
modèle, remplace le corps des fonctions `generate_voice`, `animate_face` et
`render_video` plus bas.
```

```
"""
```

```
import asyncio
```

```
import time
```

```
import uuid
```

```
from pathlib import Path
```

```
from fastapi import BackgroundTasks, FastAPI, File, Form, Header, HTTPException,
```

```
from fastapi.responses import FileResponse
```

```
from fastapi.staticfiles import StaticFiles
```

```
MAX_PHOTO_BYTES = 10 * 1024 * 1024
```

```
MAX_TEXT_CHARS = 300
```

```
RETENTION_DAYS = 30
```

```
VOICES = {"claire", "malik", "ines"}
```

```
STYLES = {"realiste", "cartoon", "3d"}
```

```
PROGRESS = {"en_attente": 0, "voix": 25, "animation": 55, "rendu": 85, "termine":
```

```
ROOT = Path(__file__).parent
```

```
DATA = Path("data")
```

```
PHOTOS = DATA / "photos"
```

```
VIDEOS = DATA / "videos"
```

```
for folder in (PHOTOS, VIDEOS):
```

```
    folder.mkdir(parents=True, exist_ok=True)
```

```
# Stockage en mémoire : remplace par une base de données (PostgreSQL, SQLite...).
```

```
jobs: dict[str, dict] = {}
```

```
app = FastAPI(title="Voxface API")
```

```
app.mount("/files", StaticFiles(directory=DATA), name="files")
```

```
class PipelineError(Exception):
```

```
    """Erreur expliquée à l'utilisateur, avec ce qu'il peut faire pour la corrige
```

```
# ----- Étapes IA (simulées) -----
```

```
def detect_single_face(photo_path: Path) -> bool:
```

```
    # TODO : détecteur de visage réel (MediaPipe, InsightFace...). Doit refuser 0
```

```

    return True

async def generate_voice(text: str, voice: str, job_id: str) -> Path:
    # TODO : appeler une API de synthèse vocale et enregistrer l'audio.
    await asyncio.sleep(1.5)
    return DATA / f"{job_id}.wav"

async def animate_face(photo_path: Path, audio_path: Path, style: str) -> Path:
    # TODO : appeler un modèle « talking head » (photo + audio vers vidéo du visa
    await asyncio.sleep(2.5)
    return audio_path

async def render_video(animation: Path, job_id: str) -> Path:
    # TODO : assembler le MP4 final (ffmpeg), avec filigrane « généré par IA » si
    await asyncio.sleep(1.5)
    out = VIDEOS / f"{job_id}.mp4"
    out.write_bytes(b"video simulée") # fichier factice, pas un vrai MP4
    return out

async def run_pipeline(job_id: str) -> None:
    job = jobs[job_id]
    try:
        if not detect_single_face(Path(job["photo_path"])):
            raise PipelineError("Aucun visage unique détecté. Essaie une photo de
        job["status"] = "voix"
        audio = await generate_voice(job["text"], job["voice"], job_id)
        job["status"] = "animation"
        animation = await animate_face(Path(job["photo_path"]), audio, job["style
        job["status"] = "rendu"
        video = await render_video(animation, job_id)
        job["video_url"] = f"/files/videos/{video.name}"
        job["status"] = "termine"
    except PipelineError as e:
        job["status"], job["error"] = "echec", str(e)
    except Exception:
        job["status"] = "echec"
        job["error"] = "La génération a échoué. Réessaie dans quelques minutes."

# ----- Routes -----

@app.get("/", include_in_schema=False)
def home():

```

```

    return FileResponse(ROOT / "index.html")

@app.get("/manifest.json", include_in_schema=False)
def manifest():
    return FileResponse(ROOT / "manifest.json", media_type="application/manifest+json")

@app.get("/icon.png", include_in_schema=False)
def icon():
    return FileResponse(ROOT / "icon.png")

def public(job: dict) -> dict:
    return {
        "job_id": job["id"],
        "status": job["status"],
        "progress": PROGRESS[job["status"]],
        "video_url": job.get("video_url"),
        "error": job.get("error"),
        "created_at": job["created_at"],
        "expires_at": job["expires_at"],
    }

def get_owned_job(job_id: str, user_id: str) -> dict:
    job = jobs.get(job_id)
    if not job or job["user_id"] != user_id:
        raise HTTPException(404, "Vidéo introuvable.")
    return job

@app.post("/avatars", status_code=202)
async def create_avatar(
    background: BackgroundTasks,
    photo: UploadFile = File(...),
    text: str = Form(...),
    voice: str = Form("claire"),
    style: str = Form("realiste"),
    consent: bool = Form(False),
    x_user_id: str = Header("demo"),
):
    if not consent:
        raise HTTPException(422, "Confirme que tu as le droit d'utiliser cette photo")
    text = text.strip()
    if not text or len(text) > MAX_TEXT_CHARS:

```

```

        raise HTTPException(422, f"Le texte doit contenir entre 1 et {MAX_TEXT_CH
if voice not in VOICES or style not in STYLES:
    raise HTTPException(422, "Voix ou style inconnu.")
if not (photo.content_type or "").startswith("image/"):
    raise HTTPException(422, "Le fichier doit être une image (JPG, PNG...).")
data = await photo.read()
if len(data) > MAX_PHOTO_BYTES:
    raise HTTPException(413, "La photo dépasse 10 Mo. Choisissez une image plus

job_id = uuid.uuid4().hex
photo_path = PHOTOS / f"{job_id}{Path(photo.filename or '').suffix or '.jpg'}
photo_path.write_bytes(data)
now = time.time()
jobs[job_id] = {
    "id": job_id, "user_id": x_user_id, "status": "en_attente",
    "text": text, "voice": voice, "style": style,
    "photo_path": str(photo_path), "consent": True,
    "created_at": now, "expires_at": now + RETENTION_DAYS * 86400,
}
background.add_task(run_pipeline, job_id)
return {"job_id": job_id}

@app.get("/avatars/{job_id}")
def get_avatar(job_id: str, x_user_id: str = Header("demo")):
    return public(get_owned_job(job_id, x_user_id))

@app.get("/avatars")
def list_avatars(x_user_id: str = Header("demo")):
    mine = [j for j in jobs.values() if j["user_id"] == x_user_id]
    return [public(j) for j in sorted(mine, key=lambda j: j["created_at"], revers

@app.delete("/avatars/{job_id}", status_code=204)
def delete_avatar(job_id: str, x_user_id: str = Header("demo")):
    job = get_owned_job(job_id, x_user_id)
    Path(job["photo_path"]).unlink(missing_ok=True)
    (VIDEOS / f"{job_id}.mp4").unlink(missing_ok=True)
    del jobs[job_id]

```